SEZIONE 2 · SOTTOSEZIONE 2.1

ARCHITETTURA, STACK ED EVOLUZIONE

2.1.1 INTRODUZIONE.

Founder's Dilemma è un business game di startup venturing per l'aula universitaria. Un solo sorgente TypeScript produce tre forme di distribuzione: app desktop Tauri, sito su GitHub Pages e file HTML unico. Il motore di simulazione è separato dall'interfaccia ed è una funzione pura di stato e decisioni (figura 2.1-1).

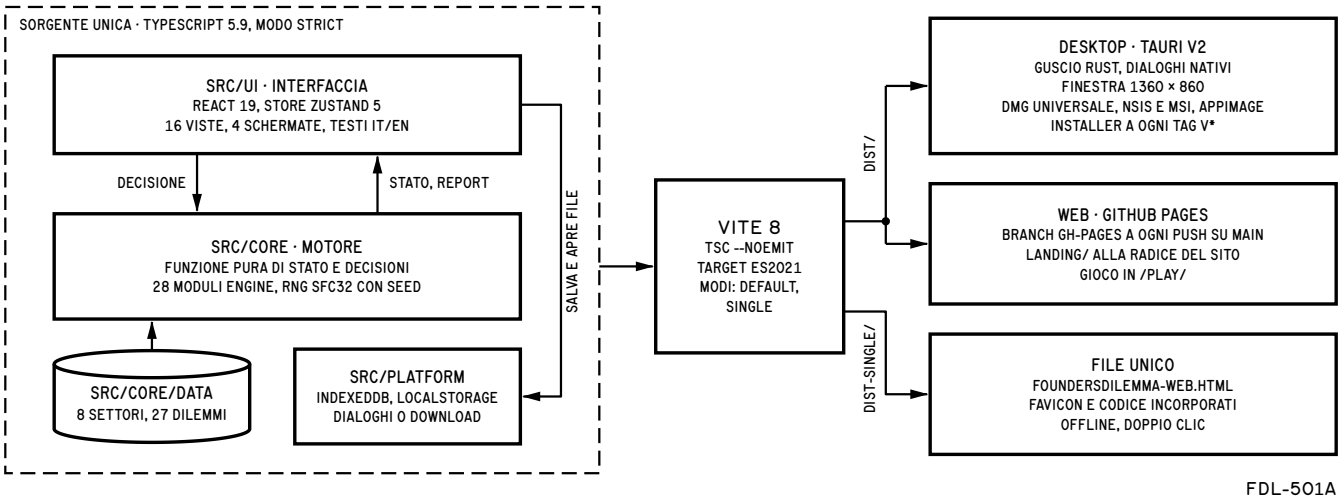


Figura 2.1-1. Schema a blocchi dello stack: un sorgente, tre forme di distribuzione

2.1.2 DESCRIZIONE FUNZIONALE.

Il motore (src/core) riceve lo stato della partita e le decisioni dei team e restituisce il nuovo stato con i report del round. L'interfaccia React legge lo stato da uno store Zustand. Il modulo src/platform salva le partite in IndexedDB e apre i file con i dialoghi nativi su desktop o con il download nel browser. Vite compila lo stesso codice in dist/ e, in modo single, in un file unico. Tre workflow GitHub Actions verificano, pubblicano e impacchettano (figura 2.1-2).

2.1.2.1 Separazione motore-interfaccia.

Nessun file di src/core importa da src/ui: lo stesso motore gira nei test Vitest in Node, nella simulazione Monte Carlo (npm run balance) e nell'app. Il guscio Rust conta 76 righe e serve solo a leggere e scrivere i file scelti dall'utente.

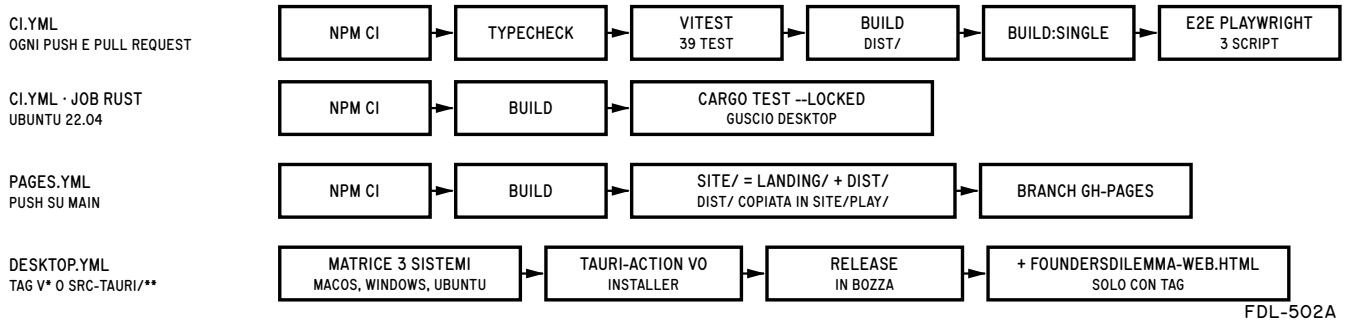


Figura 2.1-2. Flussi GitHub Actions di verifica, pubblicazione e rilascio

2.1.3 DATI DI PRESTAZIONE E DI PROGETTO.

Tabella 2.1-1. Componenti dello stack (versione 2.1.0)

Parametro	Valore
Linguaggio di motore e interfaccia	TypeScript 5.9, strict
Interfaccia e stato	React 19.3, Zustand 5
Build e test	Vite 8, Vitest 5, Playwright 1.64
Guscio desktop	Tauri 2, Rust 1.77.2 o succ.
Ambiente di sviluppo	Node.js 22, porta 1420
Finestra desktop (minima)	1360 × 860 (1024 × 680)
Righe TypeScript, motore / UI	9.922 / 12.815
Salvataggi locali	IndexedDB, store "kv"

ATTENZIONE

Il guscio desktop legge e scrive solo file .json, .csv, .txt e .html fino a 50 MB. Ogni altra estensione è rifiutata con "Unsupported file type".

NUOVA VERSIONE **MANUTENTORE**

```
1 package.json - versione
2 Cargo.toml, tauri.conf - allineare
3 TAG v2.x.y - creare, push
  √ DESKTOP APPS - in corso
4 RELEASE BOZZA - verificare
  √ FILE WEB - allegato
5 RELEASE - pubblicare
```

FDL